

Document Number: P2678R0

Date: 2022-10-13

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2022-06-22 through 2022-09-28

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [June 22nd, 2022](#)
- [July 27th, 2022](#)
- [August 24th, 2022](#)
- [September 14th, 2022](#)
- [September 28th, 2022](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)
- [P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26](#)
- [P2512: SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15](#)
- [P2605: SG16: Unicode meeting summaries 2022-01-12 through 2022-06-08](#)

June 22nd, 2022

Draft agenda:

- Continue discussion of survey questions for the 2023 C++ Developer Survey.

- Revise, add, and remove questions from the [draft survey document](#).

Attendees:

- Hubert Tong
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann

Meeting summary:

- Continue discussion of survey questions for the 2023 C++ Developer Survey:
 - *[Editor's note: The active revision at the start of the meeting can be viewed by selecting File | Version history | See version history, then selecting the version named "pre 2022-06-22 meeting", then clicking the rightward facing triangle next to the version name to "expand detailed versions"; this latter step is necessary to exclude detailed edits that otherwise interfere with numbering of the questions.]*
 - Tom asked attendees to nominate questions to be removed from consideration.
 - PBrett suggested removing Q1 (What character encoding(s) do you use for source files?) since we already have consensus for moving towards UTF-8 encoded source files.
 - PBrett asked how answers to Q1 would affect our decision making.
 - Jens concurred and asked hypothetically whether responses would entice us to, for example, add a translation phase 1 option to support GB18030 as we are doing for UTF-8 via [P2295 \(Support for UTF-8 as a portable source file encoding\)](#).
 - Jens noted that implementations that support non-UTF-8 source files will continue to support them and argued that there is nothing to be done within the standard.
 - Hubert suggested an alternative formulation that asks which scripts programmers are using in their source files and for which they might be using specific encodings.
 - Jens noted that [P2528 \(C++ Identifier Security using Unicode Standard Annex 39\)](#) assumes that everyone is using Unicode for their source file encoding and that encoding does not imply which scripts are being used.
 - Jens stated that use of a particular encoding such as ISO8859-1 does restrict what scripts can be used and that such information could potentially be used in confusability analysis.
 - Jens suggested the question could probe which scripts are used in conjunction with a non-Unicode encoding.
 - PBrett noted the existence of the Big-5 encoding and that it is being phased out in favor of GB18030 and UTF-8.
 - PBrett asked if we are at risk of discussing whether support for additional encodings should be mandated.

- Hubert responded negatively and stated that the question is intended to probe the extent to which substantial use of non-Unicode encodings remains.
- Tom stated that it sounds like we have not identified a use case for this question.
- Tom struck Q1 from the draft document.
- PBrett expressed uncertainty as to what Q2 (What character encoding(s) do you use for string literals?) is intended to ask and stated that it might be interpreted as asking if `L`, `u8`, `u`, or `U` prefixed literals are being used.
- Tom replied that the question is intended to ascertain what encodings are being used for the encoding of ordinary (non-prefixed) literals in order to learn about trends occurring in the ecosystem.
- Hubert noted that we now assume that if string literals are UTF-8, then the locale encoding is as well.
- PBrett expressed a feeling of persistent saltiness over that assumption.
- Jens stated that only `std::format` is currently pushing us towards Unicode in this way.
- Tom stated that we seem to have no use case for this question.
- Tom struck Q2 from the draft document.
- PBrett suggested removing Q10 (How are the project(s) that you work on organized for Unicode normalization?) on the basis that few programmers are aware of Unicode normalization.
- Tom responded that the question is intended to provide input regarding whether normalization should be reflected in the type system.
- Steve stated that it doesn't matter for most programmers, but that it matters immensely for a few.
- PBrett suggested it is not a good candidate question if we believe it impacts few programmers.
- Tom struck Q10 from the draft document.
- PBrett opined that Q13 (Do your project(s) use regular expressions for which the search pattern is not known at compile-time?) is important to determine if programmers create regular expressions using user input.
- PBrett stated that it probes whether [CTRE](#) is a suitable replacement for `std::regex`.
- PBrett stated that Q14 (Which regular expression languages do you use?) appears to duplicate Q12 (What libraries do you use for regular expression support?).
- Tom replied that Q14 is intended to ask which regular expression languages are being used; for example, which of the six languages supported by `std::regex` are being used.
- Hubert stated that Q12 could be useful to determine whether collation support is useful and noted that use of POSIX languages may imply better locale support needs.
- Jens observed that programmers might use those languages for other reasons.
- PBrett replied that programmers tend to use whatever language the regular expression facility they are already using supports.
- Tom struck Q14 from the draft document.

- Jens asserted that Q15 (Do you use the signed char or unsigned char types for text processing?) is not interesting.
- Hubert asked if that concern is motivated by the lack of standard library support.
- Jens replied that `iostream` supports signed and unsigned char types.
- Tom stated that the question is intended to help determine whether these types should be used exclusively as small integer types as opposed to character types.
- Jens opined that programmers should use `char`, `char8_t`, etc... for character types.
- PBrett noted that `unsigned char` is commonly used as a character type in C.
- Tom stated that this reflects a policy issue regarding whether we intend to extend the standard library to support use of these types for text and stated we have no such intent.
- Jens agreed, noted that the aliasing is unfortunate, and expressed support for not making the situation worse.
- Tom struck Q15 from the draft document.
- Jens expressed support for asking programmers how they support internationalization and localization.
- PBrett suggested dropping Q19 (What libraries do you use for collation?).
- Jens countered with a suggestion to merge Q17 (What libraries or operating system features do you use for language translation?), Q18 (What libraries do you use for localization?), and Q19.
- Tom agreed to do so.
- Tom pondered whether it is worth asking about prohibition of standard library facilities.
- PBrett responded that we can infer avoidance of the standard library when programmers state that they use, for example, ICU, but not the standard library facilities.
- Steve stated that the explicit locale capabilities present in `std::format` are representative of what programmers want.
- PBrett asked about adding a free form field for programmers to state how they support localization.
- Tom responded that it is difficult to extract data from free form entries.
- Steve stated that it is useful to know that no one uses, for example, `stdcoll()`.
- Tom asked if the "discourage or prohibit" language should be retained.
- Jens replied negatively and stated that we want to know what they do use.
- Hubert stated that Q16 (Do you use the C and C++ locale features?) is useful to know if, or to what extent, programmers depend on the C and C++ locale for identification purposes.
- Tom agreed to simplify Q16.
- Tom pondered what we would use the responses to questions about languages and scripts for.
- PBrett replied that Visual Studio Code has [UAX#9 HL4](#) features intended to help with display of bidirectional text in source files; that information could be used for SG15 guidance.

- Jens stated that the standard allows identifiers, literals, and comments to be written in many kinds of scripts; support for languages such as Japanese is intentional.
- Jens added that he favors developing guidelines to encourage features like those that Visual Studio Code offers.
- Tom noted that guidance will be forthcoming from the Unicode Source Code Ad-Hoc Group.
- PBrett concluded that it sounds like we already know we want to support these features; the data could help establish urgency.
- Jens agreed, but noted that implementors can decide for themselves what is and is not urgent.
- Tom struck Q3 and Q4 from the draft document.
- Tom opined that Q5 (Do you use characters other than the basic character set in identifiers) is probably irrelevant following the adoption of [P1949 \(C++ Identifier Syntax using Unicode Standard Annex 31\)](#).
- Steve indicated that language specific concerns are best addressed in a code style guide.
- Tom struck Q5 from the draft document.
- Discussion ensued regarding poll bias and privacy concerns.
- PBrett suggested we could ask which region of the world respondents are located in.
- Jens replied that such a question might be one that the Standard C++ Foundation is interested in asking anyway; it may not need to be included within our quota of questions.
- Hubert suggested it would be useful to emphasize culture as opposed to geographical location.
- PBrett expressed a preference for asking which nation the respondent is in.
- Tom suggested asking respondents what their native language is.
- Jens replied negatively; there are many languages spoken in India.
- Tom proposed striking Q6 (Do the projects you work on limit locale selection in deployment environments to those that use a specific character encoding?) on the basis that mainframes aren't going away any time soon.
- Tom struck Q6 from the draft document.
- PBrett suggested merging Q7 (What libraries do you use for text processing?), Q8 (How are the project(s) that you work on organized for text processing?), and Q9 (If your project(s) convert text to and from an internal encoding, what encoding(s) are used for the internal encoding?) based on an expectation that use of framework libraries like QT sufficiently answer these questions.
- Jens noted that we already have agreement that we want utilities to convert to/from UTF-8 and possibly UTF-16.
- Tom asked for clarification that such agreement is relative to locale dependent encodings.
- Steve replied yes, but also to other specified encodings.
- PBrett asserted that these questions have already been probed by JeanHeyd.

- Tom explained that Q7 (What libraries do you use for text processing?) is really intended to ascertain what features are supported via non-standard libraries because the standard does not provide adequate support for them.
- Jens suggested asking that question instead.
- Tom agreed to rephrase Q7 accordingly.
- Jens suggested asking what text processing features people most need; whether that be transcoding, Unicode algorithms, or something else.
- Jens noted that regular expression support could be added to that list differentiated by compile-time vs run-time support.
- Steve asserted that a laundry list would be ok.
- Tom stated that the next meeting is scheduled for July 13th but that we need new papers.

July 27th, 2022

Draft agenda:

- [WG14 N3016: Unicode Length Modifiers v3](#)

Attendees:

- Eskil Steenberg
- Hubert Tong
- Jens Maurer
- Marcus Johnson
- Peter Brett
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [WG14 N3016: Unicode Length Modifiers v3](#):
 - PBrett introduced the topic and invited Marcus to present his paper.
 - Marcus discussed the motivation for the paper; the desire to be able to easily format text in a Unicode encoding.
 - Tom provided a summary of the WG14 review of the paper during the recent WG14 meeting.
 - PBrett described how `gettext()` is used; a string in the string literal encoding is provided and a string in the current locale encoding is produced.
 - Tom stated that there is effectively a contract that the string produced by `gettext()` is encoded in the current locale encoding.
 - PBrett confirmed.

- PBrett asked how `printf()` would handle formatting a UTF-16 encoded argument.
- Tom replied that the existing practice for `wchar_t` based arguments is to convert them to the current locale encoding.
- Tom asked if motivation exists for an alternative behavior.
- Jens asked for an example of alternative behavior.
- Tom replied that the string literal encoding could be used to guide conversions instead of the current locale and noted that this would match the behavior chosen for `std::format()` when the string literal encoding is a Unicode encoding.
- Tom explained that such behavior would require preserving the string literal encoding for each translation unit and then somehow passing that information to `printf()`.
- Jens noted that `std::printf()` and `gettext()` have different encoding expectations; the former expects the formatting string to be in the current locale encoding while the latter expects something else.
- [Editor's note: The [GNU gettext man page](#) states:

The `msgid` argument identifies the message to be translated. By convention, it is the English version of the message, with non-ASCII characters replaced by ASCII approximations.

]

- PBrett stated that it is rare in his experience for a string literal to be passed as the format string to `printf()`.
- Victor replied that in the code base he works on, approximately 50% of `printf()` calls pass a string literal.
- Tom surmised that Victor's experience may reflect an assumption of UTF-8 as both the string literal encoding and the locale encoding.
- Victor replied that third party libraries are more likely to not assume UTF-8.
- Jens asked if there is motivation to introduce a `u8printf()`.
- Tom replied that adding such an interface is an option.
- Jens expressed belief that we have consensus that the future is UTF-8 and that transcoding operations should occur at program boundaries.
- PBrett expressed acceptance of library UB as a result of passing a format string to `printf()` that is not encoded in the expected encoding.
- Jens asked how `printf()` implementations recognize the '%' character today.
- Hubert responded that `printf()` is required to be locale sensitive and that the code point value of the '%' character may vary across encodings.
- Eskil professed that implementations simply search for a code unit that matches the ASCII encoding of '%'
- Jens argued that is an unlikely implementation choice for an EBCDIC-based system.
- Hubert explained that the '%' character encoding is non-varying across EBCDIC code pages so a simple search for a code unit that matches the EBCDIC encoding works on such systems.
- Jens surmised that, for implementations that support a locale encoding that is unrelated to the string literal encoding, there must exist a compile time decision

regarding calls to `printf()`.

- Hubert responded affirmatively and stated that the `printf()` family of functions have multiple entry points on z/OS.
- [*Editor's note: The z/OS C run-time library provides EBCDIC-based implementations and ASCII-based implementations. The latter exist to support an ASCII environment on z/OS systems. See IBM's [Enhanced ASCII support documentation](#).]*
- PBrett reported having seen cases where, if `printf()` was not locale sensitive, the results produced would not have matched expectations.
- Tom agreed that we have established that the format string must match the locale encoding.
- Eskil stated that, ideally, the string literal and locale encodings would match.
- Hubert agreed but noted that the locale encoding is controlled by the program user as opposed to the program author.
- Eskil observed that character conversions are not desirable in all cases and provided production of a JPEG header as an example.
- Jens noted that there is no current proposal to implicitly convert the `printf()` format string to the locale encoding.
- Eskil and others agreed that such a proposal would be ill-advised.
- PBrett concluded that the current `printf()` behavior matches the needs of the paper; it must already be locale encoding aware, so conversion between UTF encodings and the locale encoding is reasonable.
- Hubert agreed assuming requisite functionality as proposed in JeanHeyd's transcoding facilities.
- Hubert stated that it would be necessary to specify how transcoding errors are handled.
- Tom expressed a belief that the C standard already specifies how such errors are handled via delegation to functions like `wcrtomb()`.
- Hubert responded with a belief that the C standard requires that well-formed multibyte strings and well-formed wide strings always be interconvertible without loss.
- Tom expressed surprise that such a requirement exist.
- PBrett noted that the wording would need to specify whether the precision flag applies to code units, code points, or extended grapheme clusters (EGCs).
- PBrett stated that additional flags could select either code units, code points, or EGCs.
- PBrett asserted that the grapheme break algorithm is not too onerous a requirement.
- Tom asserted that the precision flag must specify code units for consistency with other uses of precision flags and that written code units should not split code points or EGCs.
- Hubert explained that the number of code units read from the input must not exceed the specified precision for security reasons.
- Discussion ensued regarding the possibility of buffer overflows and existing uses of the precision flag.

- Victor asked if the precision flag currently specifies the maximum number of input characters when performing wide character conversions.
- Hubert responded affirmatively but suggested verifying.
- PBrett noted that, for existing uses, code units is equivalent to characters.
- Tom explained his understanding of the precision flag; that if the precision is X , then up to X code units are read, but only the complete code unit sequences are written.
- Hubert responded that, if the input string had X code points, but the number of code units to write differs, then the same number of characters written would not match X .
- PBrett asserted that it is common to use the precision to limit output.
- Tom checked <https://cppreference.com> and reported that it claims that the `%s` specifier uses the precision to limit the maximum number of bytes to write.
- Eskil expressed a preference towards designing for the future and that legal output always be produced.
- Hubert checked the C standard and reported that the precision specifies the maximum number of output code units in the target encoding and that partial characters are not written.
- Victor summarized; the precision is the amount of output to write and the remainder of what was read is discarded.
- PBrett asserted that programmers expect the precision to express display width.
- Hubert responded that existing behavior hasn't matched that expectation for as long as multibyte encodings have existed.
- Hubert pondered whether field width has a meaning in this case.
- PBrett replied that field width fills and that precision truncates.
- PBrett asserted that what code authors really want is the ability to specify display width.
- Tom asked if there is agreement that `printf()` does not currently have the ability to specify display width.
- PBrett and Eskil responded negatively.
- Discussion ensued regarding EGCs and display width.
- Eskil expressed a preference that the C standard provide base level functionality and that additional functionality be built as libraries.
- Eskil asserted that there isn't always a single best solution.
- Hubert noted that, with regard to code points vs EGCs, splitting an EGC can produce misleading output.
- PBrett noted that virtually all programs need to interact with text in some capacity.
- Eskil stated that some capabilities are fundamental and provided the example of formatting a number.
- Eskil stated that, with regard to string types, there are uses for a size+pointer string type, a size+buffer string type, a size+capacity+buffer string type, a string-with-allocator string type, and more.
- Tom indicated that the next meeting is scheduled for August 10th and that the agenda is yet to be determined.

August 24th, 2022

Draft agenda:

- Initial planning for Kona.
- [P2626R0: charN_t incremental adoption: Casting pointers of UTF character types](#).

Attendees:

- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Initial planning for Kona.
 - Tom stated that there will likely be NB comments for SG16 to address and that they are unlikely to be available in a timeframe that would allow us to discuss them before the Kona meeting begins.
 - Tom explained that, if few people will be present in Kona, that he is inclined not to reserve a room, but rather to have both in-person and remote attendees join a Zoom meeting for discussions.
 - PBrett suggested that any such meetings should be planned for early morning Kona time in order for remote attendees in Europe and the US east coast to be able to attend.
 - Jens explained his current plans and expectations for room setup and audio capabilities.
 - Jens cautioned that the conference wifi may not handle many in-person attendees using Zoom at the same time.
- [P2626R0: charN_t incremental adoption: Casting pointers of UTF character types](#):
 - Corentin presented the paper.
 - char8_t, char16_t, and char32_t are useful for their encoding assurances, but lack support in the standard library.
 - Unfortunately, we can't just assume UTF-8 with char-based types and avoid use of the UTF variants.

- Some form of interconvertibility between `char`, `wchar_t`, and the UTF character types is needed for the latter types to be incrementally adopted.
 - Copying the content of an array of one character type to an array of another character type just because existing code needs to access it by the latter type is expensive.
 - None of the current language facilities enable zero cost interconvertibility.
 - The proposed functions are intended to have a narrow contract.
 - The names of the functions are intended to reflect the partitioning of character types that are always used with UTF data and other character types.
 - The functions are intended to provide interoperability in constant expressions.
 - The `basic_string_view` and `span` interfaces are provided for convenience.
 - The alias barrier based conversion operations that ICU uses are non-conforming, probably don't work reliably, and probably can't be made to work in the C++ core language.
 - *[Editor's note: See [SG16 issue #67](#) for more background information regarding the ICU alias barriers.]*
 - An interoperability solution is needed for the UTF character types to be adopted in practice.
- Victor asked how the proposed functions would work on a system where, for example, `wchar_t` is not the same size as `char16_t`.
 - Corentin responded that the functions are constrained such that the source and target types must have the same size and alignment; a call is ill-formed otherwise.
 - Victor requested that the paper be updated to explicitly state early in the paper what properties of the types must match for the operations to be well-formed.
 - Hubert stated that there are memory model concerns that may make this feature not worth pursuing; the proposed functions provide a very sharp feature.
 - Tom asked Corentin why he felt SG1 might want to review the paper.
 - Corentin responded that his understanding is that SG1 is generally consulted regarding the C++ abstract machine, the memory model, and concurrency concerns.
 - Jens explained that the concerns the paper raises have more to do with the object model than the memory model and that these concerns fall more under CWG than SG1.
 - Jens noted that [P2590 \(Explicit lifetime management\)](#), a paper with related concerns, was reviewed by LWG and CWG, but not by SG1.
 - Jens added that [P2590](#) completed work that began with [P0593 \(Implicit creation of objects for low-level object manipulation\)](#) and that paper also targeted LWG and CWG.
 - Corentin asked if the paper represents a good direction.
 - Hubert stated that the proposed semantics are such that, if these functions were called to replace a subobject, that the enclosing complete object would be destroyed.
 - *[Editor's note: Hubert provided a reference to the relevant wording in [\[basic.life\]p1](#) in a follow up [post to the SG16 mailing list](#).]*

- Hubert repeated his assertion that the proposed semantics have sharp edges.
- Hubert noted that there are on-going concerns involving `start_lifetime_as()` and base classes.
- Jens commented that the complete object would only be saved from destruction if there is a *provides storage* relationship ([\[intro.object\]p3](#)) between the subobject and the target type.
- Jens suggested that a better approach might be to add `constexpr` support to `start_lifetime_as_array()`.
- Jens added that it might be possible for `start_lifetime_as_array()` to offer additional guarantees in cases where an underlying type is shared.
- Tom stated that there is a complicated relationship between the core language possibilities and how that impacts the library interface possibilities.
- Tom expressed a preference for specifying an ideal library interface that then drives the core language needs.
- Hubert expressed uncertainty with regard to how to word restrictions around usage of an enclosing object following a change of type for a subobject; use or destruction of the subobject via the enclosing object would have to be avoided.
- Corentin said he would try to address that.
- Corentin stressed that, once an object's type is changed, the memory for that object cannot be accessed as though an object of the previous type is there.
- Hubert reiterated that a change of type for a subobject becomes very complicated.
- Jens asked if the paper includes examples that are reflective of how this facility would be used in something like real world code.
- Jens noted that the mailing list discussion indicated that conversion in one direction must be followed by a conversion back.
- Corentin expressed uncertainty regarding what limitations must be imposed and voiced an assumption that, since the character types are trivial, there is more flexibility.
- Jens stated that the core language has moved towards objects of a trivial type being destroyed at the same point as other types; in the past objects of a trivial type could be accessed after their point of destruction until their storage was destroyed.
- Jens noted that there may be wording that states that destruction of a trivial object where an object of another type is present results in undefined behavior and provided [\[basic.life\]p6](#) as a reference.
- Tom described his understanding of how constant evaluation works in terms of interpretation of an AST; constant evaluators can currently rely on the type system; changing the type of an object could lead to undefined behavior within the evaluator.
- Hubert agreed with Tom's description and stated that multiple implementors should be consulted.
- Corentin suggested that such problems might be avoided via dependence on an underlying type relationship.

- PBrett asked why the object type is so problematic and why, if a region of memory contains bytes that represent UTF-8 encoded text, it can't simply be accessed as an array of `char8_t`.
- Tom explained that constant evaluation is based on the C++ object model and that the concept of memory regions don't apply there.
- Corentin further explained that compiler optimizers use [type based alias analysis \(TBAA\)](#) to eliminate re-reading memory and [dead stores](#) (writes to memory that will never be observed according to the abstract machine) based on the type system.
- PBrett suggested that such alias restrictions could be removed.
- Hubert responded that doing so would impact performance.
- Jens noted that `char8_t` raised the abstraction level in C++ but not in C since `char8_t` is a type alias of `unsigned char` there.
- PBrett stated that the issue with the object model must be solved in order to specify a zero cost abstraction.
- Hubert explained that there is a trade off; using both `wchar_t` and `char16_t` increases costs, but the latter provides encoding and portability guarantees.
- PBrett opined that this suggests that use of the UTF character types is not zero cost.
- Jens responded that C++ opted to add those types as fundamental types in order to support overload resolution.
- Hubert explained the competing costs; restricting aliasing improves performance at the cost of having to workaround the type system.
- Jens noted that `memcpy()` can be used to workaround the type system.
- Tom noted that `memcpy()` can even be optimized away in some cases.
- PBrett pondered whether the abstractions adopted for UTF character types were the right choice and noted that a library facility could have provided the same encoding guarantees while using `char` internally.
- Tom explained that doing so wasn't an option for `char8_t` since UTF-8 string literals were already part of the core language.
- Steve explained that we use the type system to annotate how a block of memory is used and that `char8_t` provided the ability to annotate a block of memory as holding UTF-8 data.
- Steve asserted that making the UTF character types aliasing types would impose costs like those he has seen with code that loops over `std::byte`; the aliasing behavior hurts code generation.
- Steve noted that there are good libraries available that do use `char` and translate between code units and code points.
- Corentin stated that the choice to make `char8_t` a non-aliasing type was intentional and that any such change would further harm adoption.
- Corentin asserted that a way to use `char8_t` with historic `char`-based interfaces is needed or it just won't get used, but we'll still be left with the problems that motivated its introduction in the first place.
- Corentin opined that strong types are needed to support the [Unicode sandwich model](#).

- Corentin expressed a belief that this is solvable, implementable, and therefore should be specified.
- Jens suggested that an alternative UTF-8 design could have been based on something like `std::span<char8_t>` over a sequence of `unsigned char`.
- Jens opined that code unit types are not particularly interesting since an individual code unit by itself conveys little meaning.
- Jens noted that the proposed library interfaces have rough edges and expressed skepticism regarding a need for anything UTF specific since the underlying functionality is not encoding dependent.
- Steve agreed that the desire expressed in the paper is a special case of the problem where we want to get objects of one type out of a region of memory that holds objects of another type.
- Steve also agreed that the underlying storage for a text type is not interesting; the interface provided is.
- Steve noted that none of the suggested library solutions would have avoided the string literal concerns.
- Hubert provided a list of what he termed "a few uncomfortable facts":
 - Reading object representations is allowed but the existing wording is not satisfactory and fixing it will be hard.
 - Implementations don't always follow the standard; for example, Clang's support for placement new is non-conforming.
 - Implementations sometimes implement behavior that can't be expressed in the standard.
 - Determining that wording is sufficient requires that multiple implementations are completed based on the wording.
- Corentin, referring to earlier discussion regarding the possibility of making `start_lifetime_as_array` constexpr, noted that, since the memory location is provided by a parameter of type `void*`, any original source object type information is not present.
- Tom reported that the Unicode Source Code Ad Hoc Group suggested that SG16 author a paper to discuss the issues that have been reported following adoption of [P1949](#) for C++23 as a defect report and the migration from [immutable identifier syntax](#) to [default identifier syntax](#) in order to assist implementors with migration techniques, particularly in light of the intent for a future Unicode standard to introduce to default identifiers some currently excluded characters that are included in immutable identifiers.
 - Jens stated that he would like to understand more about the issues reported and requested that it be added to the agenda for a future meeting.
 - Hubert expressed an interest in understanding more about the discussion going on between WG21 and the Unicode Consortium.
 - Steve volunteered to add writing such a paper to his todo list.
 - Tom said he would file an SG16 issue to track the reported issues and submission of a paper.
 - *[Editor's note: Tom filed [SG16 issue #79](#).]*

- Tom stated that the next SG16 meeting is scheduled for September 14th and will likely include further discussion of [P2626R0](#) and the above requests for more information about the identifier issues and collaboration with the Unicode Consortium.

September 14th, 2022

Draft agenda:

- Report on the on-going interactions between WG21 and the Unicode Consortium.
- Report on the backward compatibility impact of [P1949 \(C++ Identifier Syntax using Unicode Standard Annex 31\)](#).
- Continued discussion of [P2626R0: charN_t incremental adoption: Casting pointers of UTF character types](#).

Attendees:

- Corentin Jabot
- Hubert Tong
- Mark Davis
- Michael Kuperstein
- Peter Bindels
- Robin Leroy
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- A round of introductions was held in honor of new attendees.
- Report on the on-going interactions between WG21 and the Unicode Consortium:
 - Tom provided an introduction and presented prepared slides.
 - *[Editor's note: Tom's slides are available at <https://github.com/sg16-unicode/sg16-meetings/blob/master/presentations/2022-09-14-WG21-UC-collab-p1949-presentation.odp>.]*
 - Unicode Message Format Working Group (MFWG):
 - Tom presented his understanding of the group's progress as previously relayed to him by Peter Brett as Peter was unable to attend the meeting.
 - Progress is on-going.
 - A draft specification is available.
 - The specification is complicated.
 - The features provided subsume those currently available in ICU.

- Implementations are available in Javascript and Rust.
 - The design might not integrate well with `std::format()`.
 - Mark elaborated on the group's work.
 - A tech preview will be available in an upcoming release of ICU; In Java first with C++ support to come later.
 - The current specification (2.0) supercedes previous work.
 - The design is intended to minimize dynamic processing.
 - In support of higher level processes, the design enables formatting to a data model that is then formatted to a string.
 - Formatting is sensitive to surrounding characters.
 - Robin stated that, with regard to dynamic and static formatting models, the previous 1.0 specification could be used to produce a statically checked implementation via code generation.
 - Michael noted that most formatting needs involve simple cases and that the interfaces provided must support difficult cases without complicating the simple cases.
 - Mark replied that making simple things simple is a goal, but that challenges naturally arise.
 - Mark provided an example of such challenges; some languages have gendered forms of sentences that should be tailored for the user.
 - Mark further emphasized the desire to cater to those cases while maintaining simplicity.
 - Tom noted an implication; that locale is insufficient by itself for producing a message; information about the recipient is needed.
 - Mark acknowledged, but noted that gender should not be imposed; formatting should reflect the diversity of recipients.
 - Michael reflected on how these concerns are expressed in social media.
 - Mark noted the concerns apply in any case where a particular user is the target of a message.
 - Mark added that western speakers are not often aware of these concerns.
- Unicode Source Code Ad Hoc Group (SCWG):
 - Tom presented the group's progress and on-going activities.
 - The group started meeting in late 2021.
 - A liaison relationship between ISO SC22 and the Unicode Consortium might be established.
 - Proposed updates to [UAX #9](#) and [UAX #31](#) were accepted for Unicode 15.
 - On-going work includes:
 - Establishing principles for source code as text.
 - Considerations for language designers.
 - A new UTS.
 - A new group will be formed to focus on issues of character confusability.

- Mark commented that the updates adopted for Unicode 15 were done to address some fairly obvious deficiencies.
- Robin categorized the updates as non-normative clarifications.
- Steve stated that [annex E](#) should be updated to reflect these clarifications.
- Steve noted such an update would only modify non-normative wording.
- Hubert cautioned that the updates must be consistent with prior intent and noted there was a desire not to speculate on uncertain interpretations at the time.
- Hubert stated that we tend to favor normative text when there is a conflict with non-normative text.
- Mark noted that non-normative text may better explain the intent of normative wording.
- Robin described in more detail some of the on-going work:
 - There will be a new UTS that will be a one-stop shop for source code.
 - Much of the focus concerns display of source code in the presence of bidirectional text or invisible characters.
 - Considerations for language design.
 - Considerations for language evolution; for example, migrating a language from immutable identifiers to default identifiers.
- Mark explained the intent to define a suite of standard profiles that language designers can choose from in order to provide a simple set of options that encompass complicated concerns.
- Corentin noted that most language designers are not qualified to determine what characters should be used for what purposes and that it is important to understand the consequences of changes.
- Corentin expressed a desire for the Unicode Consortium to make decisions about character use; for example, for what characters are allowed in an identifier.
- Mark reiterated that the goal is to make choices as easy as possible.
- Mark noted that language designers have to make choices for backward compatibility purposes and provided the example of maintaining use of '_' in identifiers.
- Mark explained that providing well-defined profiles allows language designers to better understand the implications of combining profiles.
- Mark stated that some profiles will offer the option of removing characters that are otherwise in a default included set.
- Robin acknowledged Corentin's concern and agreed with not wanting language designers to be burdened with having to consider individual characters.
- Robin stated that characters in these profiles won't be added to `XID_start` and `XID_continue` because those properties are required to be universal.
- Tom noted that this work was partially motivated by the C++ migration from immutable identifiers to default identifiers and the effort required to appreciate the consequences.

- Mark reflected on the difficulties encountered by backward incompatible changes made for XML 2.0 relating to C1 control characters.
- Robin offered assurances that a new UAX #31 revision will make the consequences of such choices more clear.
- Steve noted limitations imposed by concerns we don't have control over and provided the examples of separate compilation and linkers; identifiers might be written in normalization form C (NFC) but a linker might just interpret it as a sequence of bytes.
- Mark responded that requiring NFC is a good solution for a lot of matching cases that also arise outside of programming languages.
- Robin lamented the problems that occur by burdening users with NFC requirements and asserted that programmers can help.
- Steve noted that programs can validate NFC quickly.
- Mark agreed and noted that hits to the slow path during NFC validation are infrequent.
- Tom stated that the Unicode Consortium will form a new group to address character confusability in order to take that security burden off the programmer.
- Mark responded that the Unicode Standard provides some data regarding confusable characters but is limited to cases where glyphs for a single code point might be confused with a sequence of multiple code points; maps between code point sequences are not currently provided.
- Mark noted that confusability is often dependent on the font being used, that programming languages tend to use a reduced set of characters, and that programmers tend to use fonts that avoid some confusability issues.
- Robin explained that major changes to confusability analysis will be handled by the new group and that smaller issues will likely follow the existing processes.
- Michael asked if the confusability work will focus more on usability or security.
- Mark responded that both are important and that improving one often helps with the other.
- Corentin mentioned that visual markup for confusability can impact usability and noted that VS Code currently highlights all non-ASCII characters that might be confused with an ASCII character.
- *[Editor's note: Following the meeting, Robin Leroy shared an example of current VS Code highlighting as exhibited by Compiler Explorer (Compiler Explorer uses VS Code as its editor). The example code contains Russian text and many of the characters in that text are highlighted as confusable characters despite the surrounding context. The highlighting creates significant distraction that makes the text difficult to read. See <https://gcc.godbolt.org/z/zK7GPo9hW>.]*
- Mark acknowledged the concern and stated that efforts will be focused on avoiding markup that isn't helpful.
- Robin commented that he has a note in his working draft that states "don't do what VS Code does".

- Mark suggested a thought exercise; imagine using an editor that highlights all Latin characters that look like characters in other languages.
 - Robin explained that mixed script identifier support is important and provided `HTTP3апpос` as an example in which an identifier is composed of names that originate from different languages.
 - *[Editor's note: HTTP3апpос can be translated as HTTPRequest.]*
 - Michael expressed support for a code library that provides confusability analysis.
 - Mark replied that ICU provides confusability data but noted that application of that data necessarily requires understanding text structure.
- Report on the backward compatibility impact of [P1949 \(C++ Identifier Syntax using Unicode Standard Annex 31\)](#):
 - Tom provided an introduction.
 - [SG16 issue #79](#) tracks reports of backward compatibility impact.
 - [Clang defect report #54732](#) tracks Clang user reports; four users have reported impact, but the number of projects represented by them is unknown.
 - Robin maintains code that was impacted.
 - Robin conducted a survey of character usage in identifiers and published [L2/22-102 \(A survey of non-XID identifier usage in program text\)](#).
 - Robin explained that his code that was impacted is in a hobby project.
 - Robin described the survey he conducted and reported that it identified impacted code in a number of projects.
 - Robin reported that the SCWG intends to provide standard profiles for optional inclusion of select mathematical symbols and emoji in identifiers.
 - Robin noted that the main character differences between immutable and default identifiers is the selection of allowed mathematical symbols and emoji characters.
 - Corentin expressed concern that, if C++ were to add support for user-defined operators as Swift did, we don't want to end up in a situation where characters previously allowed in identifiers become candidates for use as operators.
 - Robin reiterated that there is no intent to add these characters to `XID_Start` or `XID_Continue`; that they are only being considered for standard profiles.
 - Robin reported that the rationale for the proposed mathematical notation standard profile for default identifiers considers existing use in languages such as Julia and Swift that support user-defined operators.
 - Robin stated that relevant experts from other members of the Unicode Consortium are reviewing that rationale.
 - Steve expressed sympathy towards use of mathematical symbols in Mathematica and that doing similarly in C++ means using those symbols in identifiers since algorithms are typically implemented as functions in C++.
 - Steve stated that the subscript and superscript characters are problematic since many fonts don't support those characters.
 - Michael asked what motivates programmers to want their code to look like mathematical equations.

- Steve responded that, in mathematics heavy fields like physics simulation, it is desirable for the code to match equations in other documents.
- Michael expressed uncertainty whether that is reasonable and reported that his closest experience has involved equations in Mathematica.
- Michael noted that typesetting languages like TeX are able to render such characters appropriately but that he wasn't sure about common programming language editors.
- Steve responded that such concerns may be limited if code is not widely shared or reused.
- Steve asserted that depending on a finicky environment is ill-advised.
- Corentin expressed a belief that language designers don't want to make such decisions and that implementors should not offer such extensions.
- Tom responded that different recommendations are appropriate for, for example, general purpose languages vs domain specific ones.
- Corentin agreed.
- Steve stated that defining standard profiles helps to provide sensible options.
- Steve suggested that profiles also provide a clearly defined feature for which implementors can be lobbied for an extension that could then be standardized based on adoption.
- Hubert replied that common extensions are not necessarily good evidence of widely used or appreciated extensions.
- Steve agreed with not wanting to make decisions on individual characters; that an appeal to authority is desired.
- Robin agreed with not placing the burden of evaluating individual characters on language designers.
- Corentin asked about the anticipated timeline for this work.
- Robin responded that a draft is expected in November, that feedback from the UTC will then be provided, and that the work is targeting next September's Unicode release.
- [P2626R0: charN_t incremental adoption: Casting pointers of UTF character types](#):
 - Tom apologized for the lack of time available to continue discussion of this paper.
- Tom stated that the next meeting will be held on September 28th and asked for opinions regarding what to prioritize next.
 - Corentin replied that continued discussion of P2626 is not a high priority right now.
 - Corentin stated that there is a need to update the standard to use and reference the current Unicode version.
 - Corentin stated that work is needed to improve estimated field widths.
 - Corentin stated that the escape string format added via [P2286 \(Formatting Ranges\)](#) needs additional work to handle combining characters in extended grapheme clusters.
 - Hubert cautioned that concern is warranted regarding debug strings getting corrupted during copy/paste operations.
 - Steve stated that Bloomberg will be filing an NB comment to update annex E.

- Hubert stated that he will be filing an NB comment about `std::format()` debug strings.
- Tom pondered the possibility of requesting that NB comment authors send copies of relevant NB comments to us when they submit them so that we can start work on them sooner.
- *[Editor's note: Tom reached out to Herb and he arranged for all SGs to get early access to NB comments.]*
- Tom reported that the next meeting will focus on LWG issues and that the following meeting will likely include a presentation from Michael.

September 28th, 2022

Draft agenda:

- [LWG #3767: `codecvt<charN\_t, char8\_t, mbstate\_t>` incorrectly added to locale](#)
- [LWG #3412: `§\[format.string.std\]` references to "Unicode encoding" unclear](#)
- Handling ill-formed Unicode in the library
 - See prior [mailing list discussion](#).

Attendees:

- Hubert Tong
- Jens Maurer
- Mark de Wever
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [LWG #3767: `codecvt<charN\_t, char8\_t, mbstate\_t>` incorrectly added to locale](#):
 - Victor provided an introduction.
 - There are four `std::codecvt` facets specified for `std::locale` that are not intended to be locale dependent.
 - This appears to be the result of an oversight; when `char16_t` and `char32_t` were added, new specializations were presumably added to match the existing `char` and `wchar_t` ones but are not actually locale dependent.
 - When `char8_t` was added, new specializations that convert between `char16_t/char32_t` and `char8_t` were added and the old specializations were deprecated.

- The overhead of the unnecessary facets is probably minimal.
 - The presence of the unnecessary facets is confusing from a design perspective.
 - The proposed resolution removes the specializations that are not actually locale dependent from `std::locale`.
 - The proposed resolution also makes the `std::codecvt` constructors publicly accessible so that specializations can be constructed without declaring derived classes.
- PBrett stated that the [email that announced the meeting agenda](#) noted that it would be helpful to understand what overhead is imposed by these additional facets in practice and asked if it had been measured.
 - Victor replied that he had not measured and that the design ramifications were of more concern to him.
 - Victor volunteered to perform some measurements and described how implementations manage the facets; via a dynamically allocated array.
 - Tom responded with his understanding that at least some implementations statically allocate the facets and just register pointers.
 - Steve asked if the proposed changes would cause existing programs to break at run-time.
 - Victor replied that the presence of the facets can be queried at run-time.
 - Tom stated an expectation that, for some implementations, complete removal of these specializations might result in link failures.
 - Steve expressed appreciation for the desire to remove these facets based on them not actually being locale dependent.
 - Victor suggested that these facets could be deprecated instead.
 - PBrett asked if the `std::codecvt` destructor should be virtual.
 - Victor expressed an expectation that a virtual destructor is inherited from a base class.
 - PBrett asserted the destructor should be declared with `override` in that case.
 - Hubert opined that these questions are more of a concern for LEWG and do not fall under SG16's purview.
 - Jens suggested an SG16 perspective that these facets are not locale dependent and therefore should not vary by locale.
 - Jens noted that these facets have been present for more than one standard cycle and removal could result in silent behavior change.
 - Jens asserted that experience should be obtained regarding the effects of removal before moving forward with a change.
 - Jens noted that those removal effects are LEWG concerns.
 - Victor agreed regarding SG16 scope for concerns.
 - Victor volunteered to investigate what the consequences of removal would be.
 - **Poll 1: SG16 agrees that the `codecvt` facets mentioned in LWG3767 "`codecvt<charN_t, char8_t, mbstate_t>` incorrectly added to locale" are intended to be invariant with respect to locale.**

- **Attendance: 7**

- **SF F N A SA**

- 4 3 0 0 0**

- **Consensus: unanimously in favor.**

- [LWG #3412: §\[format.string.std\] references to "Unicode encoding" unclear:](#)

- Hubert explained that the term "Unicode encoding" is used in several places in the standard, but with no formal definition.
- Tom provided two perspectives:
 - "Unicode encoding" refers to only those encodings specified by the Unicode standard and ISO/IEC 10646; UTF-8, UTF-16, and UTF-32.
 - "Unicode encoding" refers to any encoding that maps the entirety of the Unicode code space and therefore includes, for example, UTF-7 and UTF-EBCDIC in addition to UTF-8, UTF-16, and UTF-32.
- PBrett asked if there is an industry term that describes the latter perspective.
- Hubert replied that he is not aware of one.
- Tom replied that he had briefly looked for one in the Unicode standard when drafting the agenda email but did not find one.
- Hubert stated that, for the debug formatting output introduced by [P2286 \(Formatting Ranges\)](#), that a stateless encoding was assumed.
- Tom expressed support for restricting "Unicode encoding" to just those encodings that are defined in the Unicode Standard.
- Tom noted that, if motivation arises to support additional encodings as Unicode encodings, that a paper can argue for relaxing the restrictions.
- **Poll 2: SG16 recommends that LWG3412 "§[format.string.std] references to 'Unicode encoding' unclear" should be resolved by replacing references to "Unicode encoding" with "UCS encoding scheme".**

- **Attendance: 7**

- **SF F N A SA**

- 2 5 0 0 0**

- **Consensus: unanimously in favor.**

- Tom asked Hubert if he would be willing to research other uses of "Unicode encoding" to see if they should be similarly changed.
- Hubert agreed to do so and to open new LWG issues as appropriate.
- Jens suggested that a proposed resolution can address all such issues.
- PBrett raised concern about use of GB18030 with `std::print()`.
- Hubert noted that we don't currently use the "Unicode encoding" terminology in conjunction with `std::print()`.
- *[Editor's note: Overloads of `std::print()` for `wchar_t` and other character types are not currently provided; the wording in [\[print.fun\]p2](#) currently restricts the enhanced Unicode behavior to UTF-8.]*
- Hubert suggested we proceed with the pragmatic solution for now.
- Tom noted that, for GB18030, the latest version no longer requires use of the Unicode Private Use Area (PUA), and is therefore more likely to be considered

acceptable as a "Unicode encoding" in the colloquial sense.

- Tom stated that the issues are likely sufficiently complicated though that inclusion via a new paper is justified.

- Handling ill-formed Unicode in the library:

- Mark summarized the two issues raised during prior [mailing list discussion](#):
 - One of the examples in [\[format.string.escaped\]p3](#) is incorrect; s5 should have a result value of `["\x{c3}("]`, not `["\x{c3}\x{28}"]`.
 - It is not specified how ill-formed code unit sequences should be handled for purposes of width estimation and formatting of debug output.
- Victor responded that, for debug format output, the goal is to avoid loss of information but that concern doesn't apply to width estimation.
- Tom stated that the issue with the example is editorial since examples are non-normative.
- PBrett suggested that the width estimation issue can be addressed via an NB comment or an LWG issue.
- Tom opined that specifying the behavior for invalid code unit sequences is reasonable.
- Victor agreed and noted that this is actually a C++20 issue.
- PBrett noted that performance overhead may be potential motivation for not specifying the behavior of ill-formed input.
- Victor responded that this concern only applies to width estimation; optimizations can still be employed.
- Jens stated that, for formatting of debug output, it is clear that the intent is not to lose information.
- Tom agreed that the intent in that case is clear and well-specified; the remaining issue is width estimation for ill-formed code unit sequences.
- Jens asked what should be displayed for such ill-formed code unit sequences.
- Tom replied that such questions depend on replacement character policy.
- Jens asserted that the width estimate should be derived from the characters that will actually be displayed.
- Victor suggested that research is needed to determine what happens in practice.
- Tom noted that the input string has to be processed to calculate the estimated width, so what terminals and such do with ill-formed code unit sequences doesn't necessarily matter.
- Victor agreed and asked if the standard specifies a replacement character.
- Tom responded that he did not think it does.
- Tom suggested that the desired resolution is probably to apply [PR-121](#) policy 2 with the Unicode replacement character substituted for the ill-formed sequence.
- Victor replied that substituting a replacement character might not be easy and might impose overhead.
- Jens suggested that the best answer might be that the estimated width is unspecified.
- Mark volunteered to file an LWG issue for further follow up.

- Tom stated that the next meeting is scheduled for October 12th and that the agenda is expected to include a presentation by Michael Kuperstein unless preempted by a need to start addressing NB comments.